PLANO B

Plataforma de resiliência urbana e organização da recuperação após eventos climáticos

Atividade de Estudo Programada (AEP)
ADS / Engenharia de Software — 4º semestre — 2026.2

ODS: 11 (Cidades e comunidades sustentáveis) e 13 (Ação climática).

Natureza do documento: concepção, arquitetura e documentação de apoio à implementação do 2º bimestre.

Transparência acadêmica: a entrega do 1º bimestre não foi realizada. Este documento foi elaborado posteriormente e não deve ser apresentado como comprovação de entrega anterior. A pontuação de aderência deve ser discutida com o professor.

Maringá, Paraná • 13 de novembro de 2026

Integrantes e responsabilidades: a preencher de acordo com a participação real.

1. Descoberta, problema e público

Após interrupções urbanas severas, moradores e estabelecimentos frequentemente precisam registrar o que ocorreu, identificar possíveis impactos sobre a rotina e organizar passos de recuperação. Sem um histórico estruturado, a situação de cada ponto e as tarefas pendentes ficam dispersas.

Público interessado

Moradores, organizações de bairro, gestores de pequenos comércios e equipes de estudos sobre resiliência. A ferramenta desenvolvida é um protótipo acadêmico, não um sistema oficial de Defesa Civil.

Origem e contexto

O tema é inspirado na experiência local de temporais e danos relatados em Maringá (PR) em novembro de 2026. Todos os registros iniciais da aplicação são cenários **fictícios**; não são um levantamento oficial das ocorrências.

Proposta e alinhamento aos ODS

ODS 11: organização comunitária de informações sobre interrupções e recuperação. **ODS 13:** estímulo ao planejamento da continuidade perante eventos meteorológicos extremos. O aplicativo não prediz desastres nem emite alertas oficiais.

Principais regras de negócio

Cada ocorrência possui um tipo, localização, descrição, severidade entre 1 e 5, data e situação. Um índice orientativo de prioridade é calculado por subtipo de ocorrência com um peso diferenciado. Cada ocorrência pode conter diversas ações de recuperação e um histórico de suas mudanças de situação.

Fluxo de valor

Registrar ocorrência → consultar situação e índice orientativo → definir ações e responsáveis → acompanhar progresso → atualizar situação → consultar histórico. A decisão operacional em uma emergência deve sempre recorrer aos canais oficiais.

2. Requisitos funcionais

Código	O sistema deve...
RF01	Cadastrar ocorrências com tipo, localização, descrição e severidade.
RF02	Consultar e listar ocorrências, incluindo filtros por tipo e situação.
RF03	Atualizar os dados e a situação de uma ocorrência existente.
RF04	Excluir uma ocorrência cadastrada e seus registros dependentes.
RF05	Classificar um índice de prioridade orientativo por tipo e severidade.
RF06	Adicionar e concluir ações de recuperação vinculadas à ocorrência.
RF07	Registrar automaticamente o histórico de alterações de situação.
RF08	Mostrar um painel com totais de ocorrências por situação.

Requisitos não funcionais

RNF01: compatibilidade com Java 21 e PostgreSQL 16; RNF02: uso de JDBC e parâmetros SQL; RNF03: interface web local e responsiva; RNF04: execução reproduzível por Docker Compose; RNF05: testes de domínio e integração isolada.

Limites do escopo

Não estão incluídos despacho de equipes, reconhecimento de risco real, autenticação, geolocalização automática, atualização meteorológica, integração governamental ou publicação pública. Esses itens exigem controles e validações adicionais.

3. Justificativa técnica e arquitetura

Escolha	Razão técnica
Java 21	Tipagem, abstração, herança, polimorfismo e sobrescrita com @Override exigidos pela rubrica.
PostgreSQL 16	Armazenamento relacional persistente, chaves estrangeiras, transações e integridade referencial.
JDBC	Permite evidenciar SQL parametrizado e a conexão real com o banco sem esconder o CRUD em um ORM.
Servidor HTTP do JDK	Interface local com poucas dependências e testes HTTP executáveis no JDK.
Camadas	Separação entre domínio, repositórios, casos de uso, páginas e entrada HTTP.
Docker Compose	Banco de demonstração reproduzível e scripts SQL versionados na pasta /database.

Arquitetura lógica

Navegador → servidor HTTP (`WebApp`) → serviço (`OcorrenciaService`) → contratos (`OcorrenciaRepository` / `AcaoRepository`) → implementação JDBC → **PostgreSQL**. As classes do domínio concentram validações e o cálculo polimórfico da prioridade.

Pilares de POO demonstrados

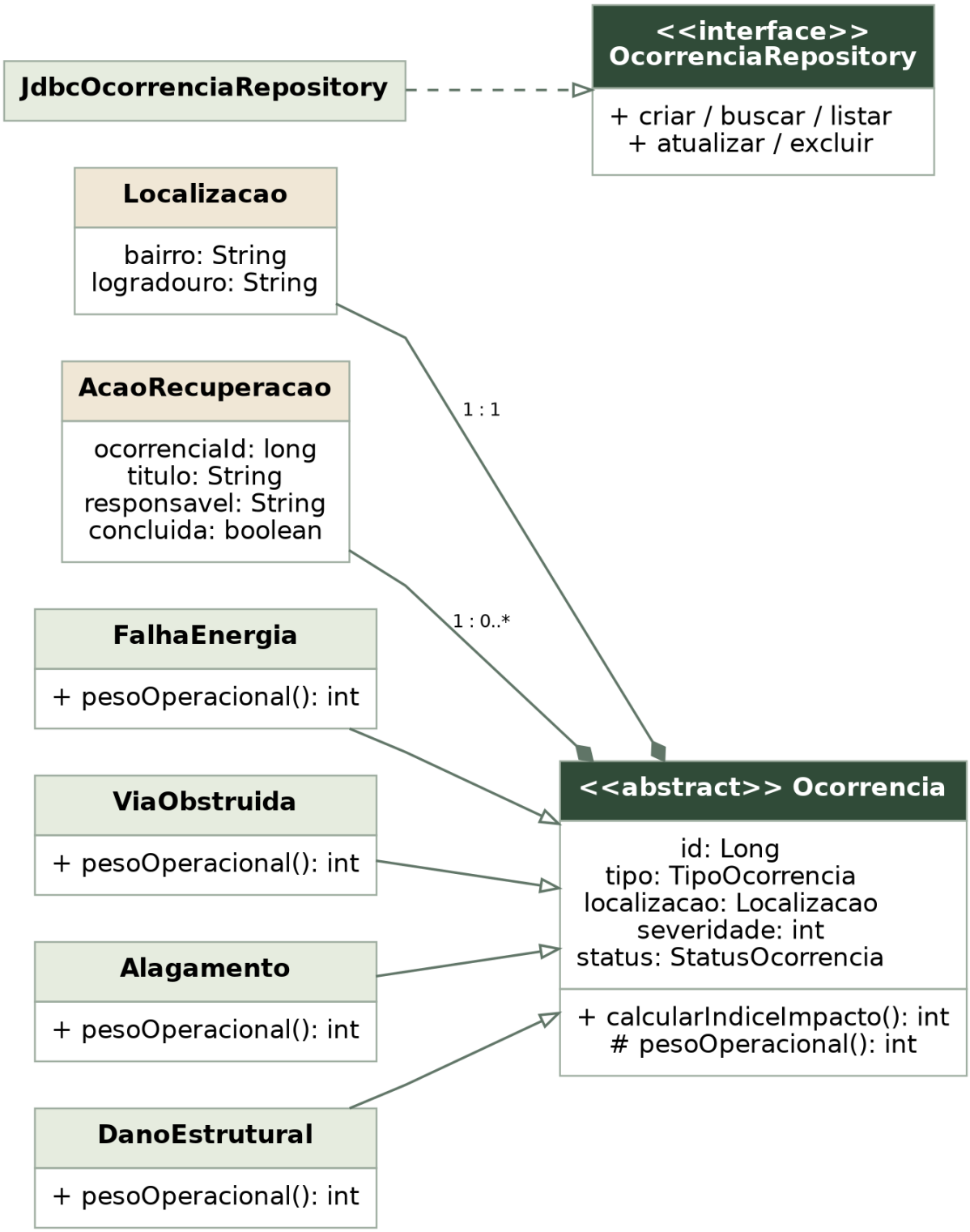
Classe abstrata: `Ocorrencia`; **Herança:** `FalhaEnergia`, `ViaObstruida`, `Alagamento` e `DanoEstrutural`; **Polimorfismo:** cada classe sobrescreve `pesoOperacional()` com `@Override`; **Interfaces:** contratos de persistência; **Composição 1:N:** uma ocorrência possui várias ações de recuperação.

Persistência e integridade

O script `database/schema.sql` define tabelas de ocorrências, ações e histórico, com chaves primárias e estrangeiras. Criar e atualizar uma ocorrência usa transação para registrar sua situação junto do histórico. Excluir ocorrência remove ações e histórico vinculados via ON DELETE CASCADE.

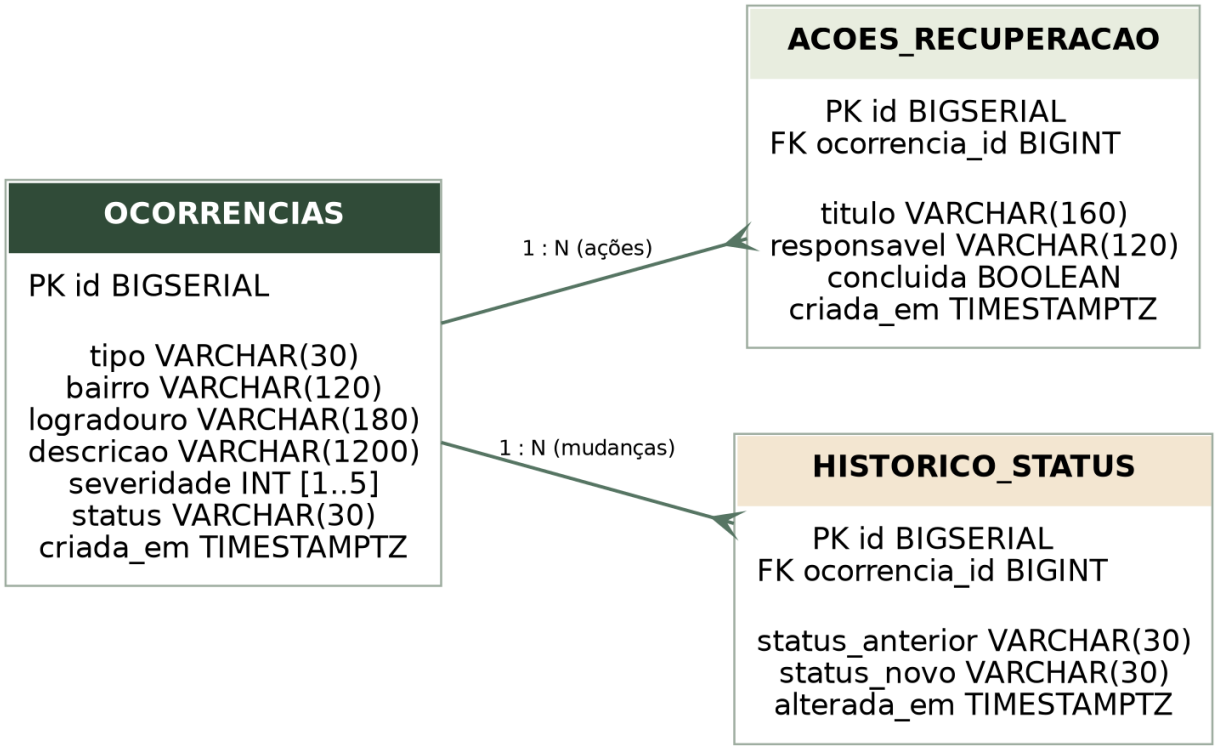
4. Diagrama de classes UML

Relacionamentos de herança, implementação de interface e composição. O método `pesoOperacional()` sobrescrito em cada subclasse demonstra polimorfismo real.



5. DER — Diagrama Entidade-Relacionamento

Modelo relacional do sistema, incluindo associações 1:N de ocorrências com ações e com alterações de situação.



A entidade principal do CRUD é `ocorrencias`. As entidades dependentes são vinculadas por `ocorrencia_id`; a integridade é implementada pelo PostgreSQL, não por listas em memória.

6. Cronograma e responsabilidades

Este cronograma foi elaborado para documentar a execução no segundo bimestre. Não corresponde a um cronograma apresentado anteriormente.

Data referência	Atividade	Responsável
13/11/2026	Escopo, levantamento de dores e requisitos	Equipe (a definir)
13/11/2026	Domínio, POO, UML e DER	Equipe (a definir)
13/11/2026	Banco de dados, CRUD e histórico	Equipe (a definir)
13/11/2026	Interface, fluxos e integração	Equipe (a definir)
13/11/2026	Testes, documentação e revisão	Equipe (a definir)

Repositório GitHub

Repositório público: https://github.com/pitado/planoB. O código foi organizado em etapas de desenvolvimento no histórico Git, sem simular participação de colegas. A primeira entrega não foi realizada.

Estrutura de repositório

`src/main/java` (domínio, serviço, JDBC e web), `src/test/java` (testes), `database` (schema e seed), `docs` (arquitetura, diagramas e relatório), `scripts` (rotinas) e `.github/workflows` (CI).

7. Validação e aderência à rubrica

Critério	Peso	Evidência / condição
Planejamento	0,2	Não houve a 1ª entrega. Critério depende de avaliação docente.
Pilares POO	0,3	Classe abstrata, interface, quatro subclasses, @Override e tipagem Java.
CRUD	0,3	Create/Read/Update/Delete na entidade ocorrência; formulários HTTP.
Banco de dados	0,2	`schema.sql`, implementações JDBC e teste integrado com PostgreSQL.

Evidências de teste disponíveis

Na data de elaboração, a compilação Java foi concluída e foram aprovadas **35 verificações de domínio e CRUD** e **24 verificações do servidor HTTP**, executadas com repositórios de teste em memória. O teste JDBC real foi implementado, mas **não executado localmente**, pois este ambiente não possui PostgreSQL e Maven prontos. Um workflow configura esse teste para execução após publicação.

Demonstração prática sugerida

Iniciar banco com Docker, executar testes, cadastrar ocorrência, alterar status, criar e concluir ação, verificar histórico, excluir registro e conferir persistência e cascata no banco. Apresentar código das classes e os diagramas ao explicar as escolhas.

Pendências para entrega acadêmica

1) Conferir o link público do GitHub e os resultados do Actions. 2) Rodar ou comprovar testes JDBC com PostgreSQL real. 3) Informar integrantes e contribuições efetivas. 4) Confirmar com o professor como o critério de planejamento será tratado sem a primeira entrega.